

System Design Mock #1 — Study Guide

Design a URL Shortener (like bit.ly)

Mock interview date: 2026-09-14

1. The Question

Design a URL shortening service like bit.ly.

Requirements pinned down during the session:

Requirement	Value
Retention	URLs stored forever, no expiry
Short key length	~7 characters (no hard limit, keep it short)
Redirect latency	p99 < ~100 ms
Write latency	< ~1 s acceptable
Traffic	~100M new URLs/day, 10:1 read-to-write ratio

2. Ideal Solution

Step 1 — Clarify requirements (do this first, every time)

Functional: - `POST /shorten` — accept a long URL, return a short URL. - `GET /{shortKey}` — redirect to the long URL (HTTP 301), 404 if unknown. - Decide: same long URL twice → same short key or different? (Pick one and justify; dedup saves storage, non-dedup skips a lookup on writes.)

Non-functional: the table above. Note this is a **read-heavy** workload — that single observation drives most of the design (caching, read replicas/scaling, DynamoDB read capacity).

Step 2 — Capacity estimation (verify your arithmetic!)

- Writes: $100\text{M} / 86,400\text{ s} \approx \mathbf{1,150\text{ writes/sec}}$
- Reads: $10\times\text{ writes} \approx \mathbf{11,500\text{ reads/sec}}$ (~12.7K total req/sec)
- Storage per URL: ~0.5 KB (short key + long URL + metadata)
- Per day: $100\text{M} \times 0.5\text{ KB} = \mathbf{\sim 50\text{ GB/day}}$
- 5 years: $50\text{ GB} \times 365 \times 5 \approx \mathbf{\sim 90\text{ TB}}$
- Bandwidth (reads): $11.5\text{K} \times 0.5\text{ KB} \approx 6\text{ MB/s}$ — trivial; latency, not bandwidth, is the constraint.

Takeaway: fits comfortably in a NoSQL KV store; the challenge is read latency at 11.5K rps, not raw size.

Step 3 — API design

```
POST /shorten
```

```
Body: { "url": "https://very-long-url..." , "custom_alias":  
"optional" }
```

```
→ 201 { "short_url": "https://bit.ly/aB3xK9p" }
```

```
GET /{shortKey}
```

```
→ 301 Moved Permanently, Location: <long URL>
```

```
→ 404 if key not found
```

- Use **301** (permanent) so browsers cache the redirect — free load reduction. (302 if you want click analytics to always hit your servers.)
- Validate input URL (scheme, blocklist) on write.
- Rate-limit `POST /shorten` per user/IP — it's the abuse vector.

Step 4 — Key generation (the heart of this question)

Key space: base62 (a–z, A–Z, 0–9), 7 chars → $62^7 \approx 3.5$ trillion keys. At 100M/day, exhaustion takes ~95 years. Fine.

Two families — know both:

A. Hash-based (deterministic): `key = base62(truncate(hash(longURL)))` -

Pros: idempotent for free (same URL → same key → conditional write), no coordination. - Cons: collisions need handling (birthday paradox: at 100M keys/day in a 3.5T space, expect collisions regularly). Handle with **salt-and-retry**: on collision, `hash(url + attempt#)` until free — each retry is an extra DB round-trip. - Self-contradiction trap: a hash can't give *two different* keys for the same URL. Pick dedup or don't, but stay consistent.

B. Counter-based (unique by construction): hand out sequential IDs, base62-encode. - Pros: zero collisions, single DB write, no retries. - Cons: needs a

distributed counter (below), and sequential IDs are **enumerable** (attacker crawls key after key — real incident class: private docs, signed URLs, reset links get harvested). - Fix enumerability with a **bijective permutation** (bit-shuffle / Feistel cipher) over the counter bits: output looks random, but one-to-one mapping means still zero collisions. YouTube video IDs work this way. (A timestamp alone does NOT fix this — timestamps are public, guessable information.)

Distributed counter without a single point of failure: don't keep one global counter. A coordinator hands out **ranges** — app server A gets IDs 1–10,000, server B gets 10,001–20,000; each serves from memory, requests a new range when dry. Coordinator can be ZooKeeper/etcd or a DB-backed allocator; it's off the hot path. Fancier alternative: **Snowflake-style IDs** (timestamp + machine ID + sequence) — no coordinator at all.

Recommended answer: counter + range allocation + bijective shuffle. One DB write per shorten, no collisions, non-enumerable.

Step 5 — Write flow

```
Client → LB → App server → [dedup check*] → Counter service (range from memory)
```

```
→ bijective shuffle → base62 → DynamoDB conditional Put (attribute_not_exists)
```

```
→ 201 { short_url }
```

- DynamoDB table: partition key = `shortKey`, attributes = `longURL`, `createdAt`, `ownerId`.
- * Dedup (optional): needs lookup by long URL → GSI on a hash of `longURL`,

or skip dedup to keep writes to a single conditional put.

- Custom aliases: check availability with the same conditional put — free.
- Spam/malware: async pipeline (SQS + workers) scans new URLs against Safe Browsing-style blocklists; don't block the write path on it.

Step 6 — Read flow (where the 11.5K rps lives)

```
Client → LB → App server → Shared cache (Redis cluster)
```

```
→ hit: 301 redirect (~1 ms)
```

```
→ miss: DynamoDB GetItem → populate cache → 301
```

- **Shared cache tier, not per-server local cache.** A load balancer sprays each hot key's traffic across N app servers; local caches duplicate the entry N times and each copy sees 1/N of the traffic (worse hit rate). One shared Redis/Memcached cluster = one copy, full traffic.
- **No invalidation needed.** URLs are immutable and never deleted → entries can't go stale. TTL + LRU eviction is the entire cache story. Say this out loud in the interview.
- **Stampede protection** for hot-key expiry (10K simultaneous misses → DynamoDB hammered):
- *Jittered TTLs* — hot keys don't expire in the same second.
- *Request coalescing / single-flight* — first miss fetches, concurrent misses wait on it.
- *Probabilistic early refresh* — refresh just before expiry.
- Partition the cache with **consistent hashing** so adding/removing nodes only remaps a fraction of keys.
- Optional: put a **CDN** in front for the hottest redirects (301s are cacheable).

Step 7 — Scaling & reliability

- App tier: **stateless** behind the LB → horizontal scaling is trivial; health checks + auto-scaling on CPU/latency.
- DynamoDB: partition key (`shortKey`) is high-cardinality and uniformly distributed (shuffled counters!) → no hot partitions. Enable auto-scaling / on-demand; multi-AZ replication is built in.
- Counter allocator: ranges make it off the hot path; run it replicated (Raft/ZooKeeper) so it's not a SPOF.
- Cache cluster: replicate + shard; a node dying just means misses (degraded latency, not outage) — DynamoDB absorbs it.
- Multi-region (if asked): writes are the hard part — either single-region writes with global reads (DynamoDB Global Tables, async replication, slight staleness on brand-new links) or accept regional shorteners.

Step 8 — Follow-ups the interviewer may throw at you

- Click analytics (per-link counters → async event pipeline, not on the redirect path).
- Custom aliases + vanity domains.
- Link expiry / edit / delete (breaks the "no invalidation" property — now you need it).
- Abuse: phishing/malware links, rate limiting, CAPTCHA on creation.
- "What if DynamoDB is down?" — serve stale from cache (URLs are immutable, stale = correct), queue writes.

Mistakes from this session — do not repeat

1. Verify estimation arithmetic before building on it (100M/day $\approx$ 1.2K writes/sec; 50 GB/day, not 10).
2. A deterministic hash can't produce two different keys for the same input — stay consistent.
3. S3 is object storage, not a low-latency KV store. Know each system's latency

profile.

4. Shared cache tier beats per-server local cache behind a load balancer.
5. Sequential IDs are enumerable — shuffle bijectively.
6. Invalidation $\neq$ eviction. Immutable data needs no invalidation.

3. Concepts Used (with interview angles)

For each concept: what it is, then how to wield it in the interview.

A. Key generation & IDs

- **Base62 encoding** — represents an integer with 62 symbols (a–z, A–Z, 0–9), much denser than decimal: 7 base62 chars hold ~3.5 trillion values vs. 10 million in 7 decimal digits. It's why short URLs look like `aB3xK9p` instead of long numbers. *Interview angle: always quantify the key space (62^7) and divide by your write rate to show exhaustion is a non-issue (~95 years at 100M/day).*
- **Birthday paradox** — collision probability grows with the *square* of item count: $\approx n^2/(2m)$ for n items in m slots. With 100M hashes/day in a 3.5T space, expect ~1,400 collisions/day — "rare" is not "never" at scale. *Interview angle: use it to justify why hash-based keys need a collision plan, and why the counter approach avoids the problem entirely.*
- **Distributed counter / range allocation** — a single global counter is a bottleneck and a single point of failure, so a coordinator leases ID *ranges* (server A gets 1–10,000, B gets 10,001–20,000); servers hand out IDs from memory and re-lease when dry. The coordinator stays off the hot path. *Interview angle: name the failure mode it solves (SPOF + contention) and the coordinator options (ZooKeeper/etcd, DB-backed allocator).*
- **Snowflake IDs** — Twitter's 64-bit scheme: timestamp + machine/datacenter ID + per-ms sequence. Unique across machines with no coordinator, roughly time-ordered (nice for DB locality). *Interview*

angle: offer it as the "no-coordinator" alternative to range allocation; note the machine-ID assignment problem it introduces.

- **Bijective permutation (Feistel cipher / bit shuffle)** — a reversible one-to-one scrambling of bits. Apply it to sequential counter values and you get random-looking keys that *cannot* collide, because one-to-one mappings preserve uniqueness. This is how YouTube video IDs work. *Interview angle: this is the textbook answer to "sequential IDs are enumerable" — say the word "bijective" and explain why it preserves collision-freedom.*
- **Random IDs + check (UUID-style)** — generate random keys, check the DB, retry on collision. Simple, no coordination — but every write pays a read, and the pool/lease variant reintroduces coordination. *Interview angle: know where this sits — fine at low scale, wasteful at 1.2K writes/sec vs. a counter.*

B. Caching

- **Cache-aside (lazy loading)** — app checks cache; on miss, reads the DB and populates the cache. Simple, and the standard pattern for read-heavy workloads like redirects. *Interview angle: state it by name — interviewers listen for the vocabulary.*
- **Read-through / write-through / write-behind / write-around** — placement of the cache relative to writes. Write-through writes cache+DB together (safe, slower); write-behind acks fast and flushes async (fast, can lose data); write-around skips the cache on write. *Interview angle: for immutable URLs, the choice barely matters — say so; it shows you match the pattern to the data lifecycle.*
- **LRU / LFU eviction** — when the cache is full, drop the least-recently-used (LRU) or least-frequently-used (LFU) entry. This is *eviction*, not invalidation: the data is still valid, you're just making room. *Interview angle: never say "invalidate" when you mean "evict" — interviewers pounce on this.*
- **TTL + jitter** — expire entries after a time-to-live; add randomness to the

TTL so thousands of hot keys don't expire in the same second. *Interview angle: jitter is a one-line stampede defense — always mention it with TTLs.*

- **The three cache failure modes** — *penetration* (queries for keys that don't exist hammer the DB — fix with negative caching), *breakdown* (a hot key expires and simultaneous misses stampede the DB), *avalanche* (many keys expire at once, e.g., after a deploy). *Interview angle: naming all three unprompted signals senior-level cache literacy.*
- **Request coalescing / single-flight** — when N requests miss on the same key simultaneously, only the first hits the DB; the rest wait on its result. *Interview angle: the standard answer to cache breakdown besides jitter.*
- **Probabilistic early refresh** — refresh a hot entry just *before* it expires (probability rising as expiry nears), so it never actually goes cold. *Interview angle: the "fancy" third stampede mitigation — mention after jitter and coalescing.*
- **Hot keys** — a tiny fraction of keys gets most traffic (viral links). They can overwhelm a single cache node even with consistent hashing. Mitigations: replicate hot keys across nodes, add a small local L1 in front of the shared cache, or detect-and-replicate. *Interview angle: connect it to your key design — shuffled counters spread hot keys uniformly, which is exactly what you want.*
- **Negative caching** — cache "not found" results (short TTL) so repeated 404s for garbage keys don't reach the DB. *Interview angle: pairs with the penetration failure mode; cheap insurance.*

C. Storage & data

- **DynamoDB partition key** — determines which physical partition stores the item; throughput scales with partitions. Pick high-cardinality, uniformly distributed keys (shuffled counters are ideal) or you get throttled hot partitions. *Interview angle: always justify your partition key choice against hot-partition risk.*
- **Global Secondary Index (GSI)** — a second "view" of the table keyed by a

different attribute, enabling lookups the base table can't do (e.g., dedup: find shortKey by longURL hash). Costs extra write capacity and storage.

Interview angle: mention the cost — GSIs aren't free, which is a real argument for skipping dedup.

- **Sharding strategies: hash vs. range** — hash sharding spreads writes uniformly but can't do range scans; range sharding supports ordered queries but risks hotspots on sequential keys. *Interview angle: your shuffled counter keys make hash sharding safe — connect the two decisions.*
- **Replication: single-leader vs. multi-leader** — single-leader: all writes go to one node, replicas follow (simple, failover needed). Multi-leader (DynamoDB Global Tables): writes accepted in multiple regions, conflicts resolved by last-write-wins (available, slightly stale). *Interview angle: for the multi-region follow-up — new links may 404 briefly in a far region under async replication; name the tradeoff.*
- **Strong vs. eventual consistency** — strong: a read always sees the latest write (costs latency/availability). Eventual: replicas converge, reads may be stale. DynamoDB GetItem defaults to eventual, ~2× cheaper. *Interview angle: for redirects, eventual is fine (URLs are immutable — stale is impossible); for the dedup check, you'd want strong.*
- **Quorum reads/writes ($R + W > N$)** — with N replicas, read from R and write to W ; if $R + W > N$, reads always overlap a node holding the latest write → strong consistency on an eventually-consistent store. *Interview angle: the 30-second "how do you get strong consistency from Dynamo-style stores" answer.*
- **Bloom filter** — a tiny probabilistic structure that answers "definitely not in set / probably in set." Put one in front of the DB to cheaply reject lookups for never-created keys (penetration defense). Small false-positive rate, zero false negatives. *Interview angle: a compact, impressive add-on for the "garbage key" attack on your redirect path.*
- **Conditional writes** — PutItem with `attribute_not_exists(pk)`: the write succeeds only if the key is absent, atomically. Gives you idempotent

creates and race-free custom-alias claims with zero extra reads. *Interview angle: name it when the interviewer asks "what if two users claim the same custom alias at once."*

D. Traffic, scaling & the web path

- **Load balancing algorithms** — round-robin (simple, assumes equal servers), least-connections (better under uneven load), consistent-hash routing (same key → same server, useful for local caches). Plus active health checks to pull dead servers out. *Interview angle: tie the algorithm to your cache choice — consistent-hash routing is the one setup where per-server local caches make sense.*
- **Stateless app tier** — servers hold no per-request state, so any server handles any request; scaling is just adding boxes behind the LB. *Interview angle: say "stateless" explicitly when describing the app tier — it's a scaling claim the interviewer wants to hear.*
- **Consistent hashing** — keys and nodes sit on a hash ring; each key maps to the next node clockwise. Adding/removing a node remaps only $\sim 1/N$ of keys instead of everything. *Interview angle: the answer to "how does your cache cluster grow without a full reshuffle."*
- **Virtual nodes** — each physical cache node claims many points on the ring, smoothing out uneven key distribution and making node add/remove even less disruptive. *Interview angle: the one-line deeper cut if they push on consistent hashing.*
- **CDN edge caching** — cache 301 redirects at edge PoPs close to users; the hottest links never reach your origin at all. 301s are cacheable by design. *Interview angle: the cheapest possible scaling for a viral link — mention it as the outer layer of the read path.*
- **301 vs. 302** — 301 (permanent): browsers and CDNs cache it, fewer hits to you, but you lose per-click visibility. 302 (temporary): every click hits your servers — required if you need click analytics. *Interview angle: frame it as a business tradeoff (cost vs. analytics), not just an HTTP trivia answer.*

- **Rate limiting: token bucket vs. leaky bucket vs. fixed window** — token bucket allows bursts up to bucket size, refills steadily (good for APIs); leaky bucket smooths to a constant rate; fixed window is simplest but lets 2× bursts at window edges (fix: sliding window). *Interview angle: put the limiter on POST/shorten — it's the abuse entry point — and name token bucket as your pick.*
- **API gateway** — single front door handling auth, rate limiting, request validation, and routing before traffic reaches services. *Interview angle: where cross-cutting concerns (auth, throttling) live instead of being reimplemented per service.*
- **Message queues (async work)** — push non-urgent work (click analytics, malware scanning of new URLs) to a queue; workers drain it at their own pace. Decouples write latency from processing time. *Interview angle: the answer to "don't block the write path" — analytics and safety scanning both go here.*
- **Connection pooling** — reuse DB/cache connections instead of opening one per request; at 12K rps, connection setup would dominate latency. *Interview angle: a small detail that shows you've operated services at scale.*
- **TLS termination** — decrypt HTTPS at the load balancer so app servers handle plain HTTP; certificates live in one place. *Interview angle: mention once in the architecture sketch; don't dwell.*

E. Reliability & operations

- **Single point of failure (SPOF) analysis** — for every component, ask "what happens when this dies?" The counter coordinator, the cache cluster, DynamoDB itself. Design so each failure degrades rather than kills. *Interview angle: volunteer a failure walkthrough before being asked — it's a senior signal.*
- **Graceful degradation** — when the cache cluster dies, redirects still work via DynamoDB (slower, not broken); when DynamoDB is down, serve from cache (immutable URLs mean stale = correct). *Interview angle: pair each*

SPOF with its degraded mode.

- **Circuit breaker** — stop calling a failing dependency after N failures and fail fast, instead of piling up timed-out requests; probe periodically to close the circuit when it recovers. *Interview angle: protects the app tier when DynamoDB has a bad minute.*
- **Retry with exponential backoff + jitter** — retry failed calls with growing delays (1s, 2s, 4s...) plus randomness so all clients don't retry in lockstep and re-thunder the recovering service. *Interview angle: "jitter" again — interviewers love hearing it twice in the right places.*
- **Health checks & auto-scaling** — LB health checks remove sick servers; auto-scaling adds capacity on CPU/latency signals. *Interview angle: how the stateless tier actually scales in practice, not just in theory.*
- **SLIs/SLOs** — SLIs are what you measure (p99 redirect latency, redirect success rate); SLOs are the targets (p99 < 100ms, 99.99% success). *Interview angle: restate the interview's latency requirement as an SLO — it frames every later decision.*
- **Observability: metrics, traces, logs** — metrics (request rates, cache hit ratio, p99s) for dashboards/alerts; distributed traces to find which hop is slow; structured logs for debugging single requests. *Interview angle: "how would you know the cache hit ratio dropped?" — metrics, and say which ones.*